Traffic Classification Using In-Kernel eBPF Features – Final Report

Adeeb Fahim, Adeolu Agboola, Husham Al-Jeddah, Djimon Watson
Carleton University
Ottawa, ON, Canada

Abstract - With increased complexity in networks, modern systems must be able to manage different types of traffic in an efficient manner. Traffic classification is a common feature that achieves this, but common implementations have several drawbacks. This paper details the use of in-kernel Linux programs accessible to eBPF and machine learning features to determine the effectiveness of traffic classification at the user space level. We also explore the results of these experiments such as traffic efficiency and accuracy. As a result, we found that using eBPF-available features and machine learning came at a small cost of network efficiency but was still extremely accurate in correctly labelling traffic. The performance trade-offs are small enough that the benefits of a user space traffic classifier may be worth pursuing depending on the use case.

Introduction

Traffic classification is a critical task within networks and provides insight into several network factors such as management, security, performance optimization, quality of service, and intrusion detection. Traditionally, this task has been performed using deep packet inspection, however there are various trade-offs associated with it, such as cost, privacy issues, and high computational overhead. This project explores a different method involving the use of eBPF, which is a lightweight mechanism that can run sandboxed programs directly in the kernel as high performance packet processing without the associated overhead of DPI, allowing users to view flow-level traffic.

Problem

Common traffic classification implementation has several issues. Simpler configurations primarily use port numbers to label traffic types. This implementation does not consider uncommon port numbers and could also be easily spoofed by malicious actors. In more complex or secure networked systems, or systems that require high classification accuracy, more deeply integrated methods to classify traffic are used, such as kernel-level classification and DPI. This introduces performance overhead to the system due to higher resource usage and increased packet delay. This report goes into the details of using methods outside the Linux kernel space to identify a method that solves or mitigates these problems while largely maintaining the high accuracy of typical classification methods.

Approach and Implementation

The overall approach of this project was to combine lightweight, in-kernel traffic observation with offline machine learning to determine if flow-related features available to eBPF are sufficient for traffic classification.

The system captures metadata and temporal features at the Linux Traffic Control (tc) ingress hook. It then exports the resulting flow records to user space, where machine learning models are applied for training and evaluation. This design does not rely on deep packet inspection and instead leverages the efficiency of kernel-space instrumentation with the flexibility of user-space analytics.

Incoming packets first arrive at a selected network interface. An eBPF program is attached at the tc ingress hook, observing each packet before it is processed further by the networking stack. For each flow, an entry is added to a BPF hash map. The key for this hash map is based on network and transport-layer metadata, while the map value contains flow statistics such as packet count, byte count, flow lifetime, and inter-packet timing. In user space, an exporter polls this map, converts the kernel-side records into structured CSV rows, and stores the dataset for training and evaluation. Machine learning models are then trained on this exported data and compared against a simpler kernel-side rule-based classifier.

The core implementation of the feature extraction stage is the eBPF flow tracker. In earlier stages of the project, a simple packet-counting program was used to validate the eBPF toolchain, attachment workflow, and BPF map visibility. Once this functionality was confirmed, the implementation evolved into a more complete flow-monitoring program based on a unified flow key. The key contains flow information such as source and destination address and port, protocol, and IP version. The associated map value stores flow metrics such as packet count, inter-packet time, and aggregated TCP flags. This representation was chosen as it captures richer flow behavior than a simple port-based solution while remaining available at the kernel hook and avoiding payload inspection.

In addition to feature extraction, the kernel program also performs lightweight rule-based classification. This is based on protocol and transport ports, allowing eBPF logic to assign traffic labels such as ICMP, HTTP, DNS, SSH, IPERF3, and QUIC. In the evaluation, this kernel-side classifier was retained as a baseline rather than the main source of model input. The extended implementation also includes threat-related data structures, such as rate-limiting counters, a block-list map, and configuration maps. These additions demonstrate how the feature-extraction pipeline could be extended to cybersecurity, but the primary evaluation in this report focuses on behavioral classification accuracy and RTT overhead.

To make kernel-side flow records usable for machine learning, we implemented a user-space export pipeline. The exporter periodically queries the flow map using bpftool, parses the raw contents, and writes the data to a CSV format. The dataset includes protocol info, IP version, addresses, ports, packet and byte counters, timing-based features, TCP flags, kernel labels, and ground-truth traffic labels for traceability and dataset construction. For the evaluation, the exporter preserved protocol and port fields in the CSV for auditability, but the evaluation script removed these columns from the model feature matrix before training and testing. Supporting scripts automate traffic generation for controlled traffic classes allowing

consistent data to be captured from the eBPF program.

Machine learning was implemented as an offline behavioral analysis pipeline. In the final experiment, the models were trained and tested only on flow-behavior features. Three classifiers were evaluated: Random Forest, Decision Tree, and k-Nearest Neighbors. The evaluation scripts generated standard classification metrics and supporting visualizations.

A monitoring and mitigation extension was developed on top of the core pipeline. A terminal-based monitoring tool provides a live view of active flows and related data. A background daemon loads the trained Random Forest model and performs periodic inference over the exported kernel flow state. This daemon can update a kernel block-list map, allowing selected traffic types to be blocked in kernel space. This component was used as an implementation extension on top of the main classification pipeline rather than as the central experimental result.

Overall, the system presents a layered implementation of the project objective. The eBPF program captures efficient in-kernel flow data, exporters convert the data into a structured dataset, and ML models analyze and classify the resulting flows using behavioral features.

Experimentation

The experimental evaluation was created to answer two questions. First, can traffic be classified correctly using behavioral flow-level features collected at an in-kernel eBPF hook, without relying on deep packet inspection or well-known port numbers? Second, how much performance overhead is introduced by the eBPF-based monitoring pipeline compared to a baseline path? To address this, we ran experiments covering behavioral dataset generation, offline machine learning evaluation, classifier comparison, and runtime overhead measurement.

The environment was based on a Linux system with eBPF enabled. TC was used as the ingress hook where the eBPF program was attached. The loopback interface was used for controlled testing to reduce environmental variability and allow repeatable traffic generation. The toolchain involved clang for eBPF compilation, bpftool and tc for program attachment and map inspection, and Python scripts for data export, model training, and result visualization.

1. Behavioral Classifier Evaluation on eBPF-Exported Features

The main evaluation experiment used a behavioral dataset generated from five controlled traffic classes: DNS, HTTP, ICMP, IPERF3, and QUIC. Each traffic type was generated separately while the eBPF program collected flow statistics at the tc ingress hook. The final dataset contained 388 traffic flows, each belonging to one of the five traffic classes.

The final feature set excluded protocol number, source port, destination port, IP addresses, kernel label, heuristic label, threat level, and the ground-truth label. The classifiers were trained only on behavioral features such as packet count, byte count, flow duration, inter-packet timing statistics, TCP flag activity, bytes per packet, packets per second, and bytes per second.

Three classifiers were evaluated: Random Forest, Decision Tree, and k-Nearest Neighbors. Model evaluation used standard machine learning metrics including precision, recall, classification accuracy, macro F1-score, training time, and inference time per flow. The script also generated image artifacts summarizing the results, including behavioral confusion matrices, feature-importance plots, model-comparison plots, and accuracy-versus-overhead plots. This experiment is the central evaluation of the project because it directly tests whether eBPF-visible behavioral features are sufficient for traffic classification without relying on protocol or port lookup.

2. Kernel-Side Versus User-Space Classifier Comparison

A second experiment compared the lightweight kernel-side classifier against the user-space behavioral machine learning classifier. The kernel classifier uses rule-based protocol and port logic inside the eBPF program, while the behavioral ML classifier uses only the reduced non-leaky feature set described above. This comparison was included to contrast two design choices: immediate in-kernel heuristic classification versus user-space classification using learned flow behavior.

The output of this experiment is summarized in a confusion matrix and bar graph which visually compares the two approaches. This comparison is useful because it captures the trade-off between simplicity and expressiveness. The kernel classifier is lightweight and available immediately at the hook point, while the ML classifier can exploit richer statistical structure in the exported eBPF features at the cost of user-space processing.

3. Runtime Overhead Evaluation with RTT Benchmarking

To evaluate the runtime cost of kernel instrumentation, an RTT benchmark was implemented using a comparison script with an eBPF RTT program. This benchmark measures round-trip latency under two controlled conditions. In the baseline phase, standard ICMP ping traffic is sent without the RTT eBPF program attached. In the instrumented phase, the same workload is repeated with the eBPF RTT measurement program attached at the tc ingress hook. This design isolates the overhead associated with the added kernel instrumentation.

The benchmark produces two main outputs for comparison. The first is a CSV file, which records the measured values for baseline and instrumented execution. The second is a bar graph which visualizes the difference in average RTT between the two conditions. In addition, the benchmark reads statistics from the BPF RTT map, allowing the report to compare user-visible ping measurements with RTT statistics gathered directly in kernel space.

This experiment addresses the second major objective of the project: determining whether eBPF-based flow monitoring introduces acceptable runtime overhead. Unlike the classifier evaluations, this benchmark quantifies the system-level cost of the monitoring approach itself.

Results

As shown in Figure 1, all the algorithms were able to effectively classify all five types of traffic, with only a few misclassifications of DNS and IPERF traffic.

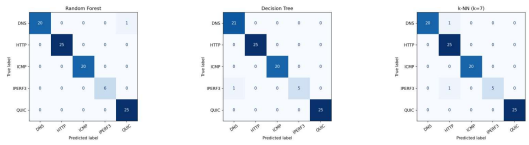

Figure 1: Confusion matrices for Random Forest, Decision Tree, and k-NN using behavioral eBPF flow features only.

Across all algorithms, classification performance remained high. Random Forest and Decision Tree achieved an identical accuracy of 98.97%, but Random Forest achieved a higher macro F1-score of 0.9923 versus Decision Tree's 0.9772. k-NN recorded 97.94% accuracy with a macro F1-score of 0.9692. This indicates that features exported from eBPF were sufficient to distinguish the traffic classes in the controlled workload.

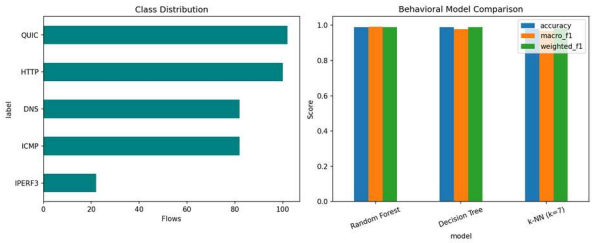

Figure 2: Comparison of behavioral classifier performance across Random Forest, Decision Tree, and k-NN.

We also compared the three algorithms the classification accuracy to the inferred latency. Random Forest provided the strongest class-balanced performance, with an accuracy of 99.12%, but introduced a high overhead of 0.1267 ms/flow. Decision Tree had the lowest inference overhead at approximately 0.0141 ms/flow, while recording 98.97% accuracy. k-NN required 0.0224 ms/flow and achieved 97.94% accuracy. Therefore, Random Forest was the strongest model in terms of accuracy, while Decision Tree was best for performance with the least latency.

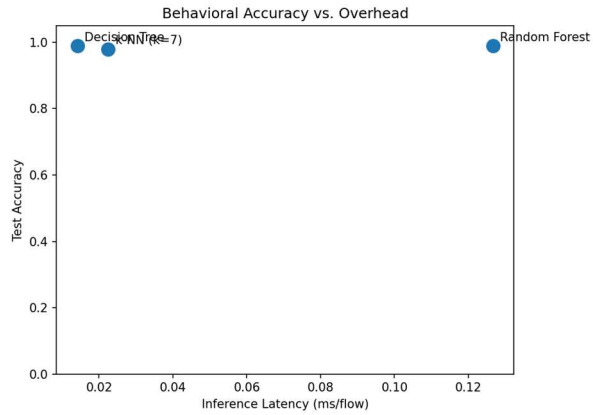

Figure 3: Accuracy versus inference overhead for the behavioral traffic classifiers.

Feature importance was also measured as shown in Figure 4. In the Random Forest model, the most important features by far were byte count and bytes per packet. Byte count had an importance of 29.16%, while bytes per packet had a score of 28.58%. These results show that the classifier primarily used traffic volume and packet-size behavior to separate traffic classes. This addresses the concern of traditional methods classifying traffic primarily with well-known port numbers.

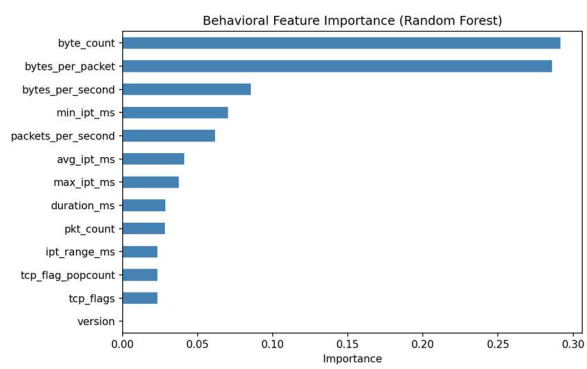

Figure 4: Random Forest feature importance after removing direct protocol and port identifiers from the model input.

The behavioral Random Forest classifier was compared against the kernel classifier. The

kernel achieved 97.94% accuracy and a macro F1-score of 0.8141, while the behavioral Random Forest classifier achieved 98.97% accuracy and a macro F1-score of 0.9912. Although the kernel classifier performed well in terms of overall accuracy, its lower macro F1-score shows less balanced performance across the traffic classes. The ML classifier provided more consistent classification across classes while avoiding direct use of protocol and port features.

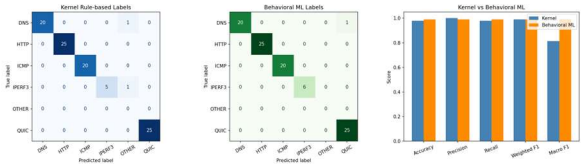

Figure 5: Kernel rule-based classification compared with the behavioral Random Forest classifier.

Finally, the RTT benchmark measured the overhead introduced by eBPF. In the benchmark, the average RTT of a baseline ping was 0.039 ms, while average RTT with eBPF attached was 0.040 ms. This corresponds to about 2.56% overhead in the ping-measured average RTT. This suggests that eBPF introduced a small runtime cost while enabling detailed in-kernel measurement.

Conclusion

Our experiments involving eBPF accessible features and machine learning evaluation were done to evaluate the viability of traffic classification programs that operate largely outside of the kernel space of a Linux system. After developing an eBPF program to capture traffic, evaluating the classifier against several performance and accuracy metrics and comparing it to the kernel-based classifier, we found that a classifier using only eBPF features can closely match the capabilities of the kernel classifier. The labelling accuracy rate came close to the perfect rate of the kernel, while the overhead introduced was minimal. The classifier took several variables of a packet into account and prioritized protocol flags over port numbers, which helps against the common issue of port number spoofing and makes classification more flexible in identifying traffic without DPI. We conclude that an eBPF-based classifier can be sufficiently accurate compared to the kernel.

Citations and Related Works

[1] Ahmad Azab et al. “Network traffic classification: Techniques, datasets, and challenges,” ScienceDirect, June 2024. Available: https://www.sciencedirect.com/science/article/pii/S2352864822001845

[2] Jorge Gallego-Madrid et al. “Machine learning-powered traffic processing in commodity hardware with eBPF” Science Direct, April 2024. Available: https://www.sciencedirect.com/science/article/pii/S1389128624001270

[3] Shane T. Miller “Decision Trees, Random Forests, and Nearest-Neighbor classifiers” Michigan Technological University, March 2, 2020 Available: https://pages.mtu.edu/~shanem/psy5220/daily/Day13/treesforestsKNN.html

[4] DataScienceBase “KNN vs Other Algorithms”, 2025, Available: https://www.datasciencebase.com/supervised-ml/algorithms/k-nearest-neighbors/comparison/